Software Re-Engineering

Lecture 1

Introduction to software re-engineering

Instructor:

Dr. Natalia Chaudhry,

Assistant Professor, Software Engineering Department, PU.

SE Fall 2020

Agenda

- Introduction to course
- Define sw re-engineering
- Motivation for sw re-engineering
- Basic concepts and preliminaries related to sw re-engineering
- Software evolution and maintenance
- Understanding the challenges of legacy projects
- Characteristics of a legacy project
- Steps in sw re-engineering
- Techniques used for re-engineering
- Software Reengineering Strategies

Introduction

Credit Hours: 3 (3,0)

Prerequisites: Software Construction and Development

Topics:

1. Introduction to Software Re-Engineering
2. Software Evolution
3. Legacy Systems
4. RE-Engineering Techniques overview (reverse engineering, restructuring & forward propagation)
5. Reverse Engineering & its techniques
6. Refactoring code to analysis artifacts
7. Refactoring code to architecture
8. Object Oriented Re-engineering Patterns
9. Program Comprehension
10. Code restructuring
11. Quality issues in re-engineering processes
12. Tool support for Re-engineering, Challenges & Stakeholder aspiration
13. Software maintenance and re-engineering economics

Software re-engineering

Software re-engineering is a process of sw development which is done to improve maintainability of a software system.

Or

Software re-engineering is the examination and alteration of a system to reconstitute it in a new form.

The process is a combination of sub-processes such as **reverse engineering, restructuring, redocumentation, forward engineering,** and **retargeting.**

Motivation for re-engineering

cost savings

- Cuts in govermment and industry budgets have reduced the resources available for the development of new software systems.
- Consequently, organizations are seeking ways to adapt current systems to accommodate new functionality to meet changing needs

Possible reduction of maintenance costs.

- In many of today's systems, maintenance changes have been directly implemented in the code and have not been carried back to the design documentation of the systems.
- Lack of documentation and complexity of code force maintainers to devote an extensive amount of time trying to understand the functions of a system.
- Reengineering provides a means of reworking the documentation and code into a more maintainable format that allows maintainers to quickly gain a better understanding of the system

Contd..

Design recovery

- to recover and record lost system information.
- This can reduce an organization's dependence on those individuals who understand the present software, and shorten the time necessary for new individuals to learn the system.

Identify "trouble-spots."

- Errors can be detected and corrected, or modifications made to improve system performance.
- The information gained during reverse engineering can be used to restructure the system, thus making the system more maintainable.

migrating the functionality of an older system to a new computing environment

reuse of software components from existing systems

Contd..

Reengineering implies a single cycle of taking an existing system and generating from it a new system, whereas evolution can go forever

Software evolution

- Software change is inevitable
  - New requirements emerge when the software is used
  - The business environment changes
  - Errors must be repaired
  - New computers and equipment is added to the system
  - The performance or reliability of the system may have to be improved.

Importance of evolution

- Organisations have huge investments in their software systems - they are critical business assets.
- To maintain the value of these assets to the business, they must be changed and updated.
- The majority of the software budget in large companies is devoted to evolving existing software rather than developing new software.

Maintenance

- Modifying a program after it has been put into use.
- Maintenance does not normally involve major changes to the system’s architecture.
- Changes are implemented by modifying existing components and adding new components to the system.

Importance of maintenance

- The system requirements are likely to change while the system is being developed because the environment is changing. Therefore a delivered system won't meet its requirements!
- Systems are tightly coupled with their environment. When a system is installed in an environment it changes that environment and therefore changes the system requirements.
- Systems MUST be maintained therefore if they are to remain useful in an environment

Evolution vs maintenance



- All support activities carried out after delivery of software are put under the category of **maintenance**.
- All activities carried out to effect changes in requirements are put under the category of **evolution**.

- Yang and Martin Ward defined software evolution as “...the **process of conducting continuous software *reengineering***

Some concepts..

Reengineering = Reverse engineering +Δ+ Forward engineering

“reverse engineering”

- The input to the reverse engineering process is the source code of the system, and the output is the system architecture.
- It is the process of examination of the system, and it is not a process of change.
- It does not involve changing the software under examination

Contd..

“forward engineering” is the traditional process of moving from a high-level abstraction to the physical implementation of the system.

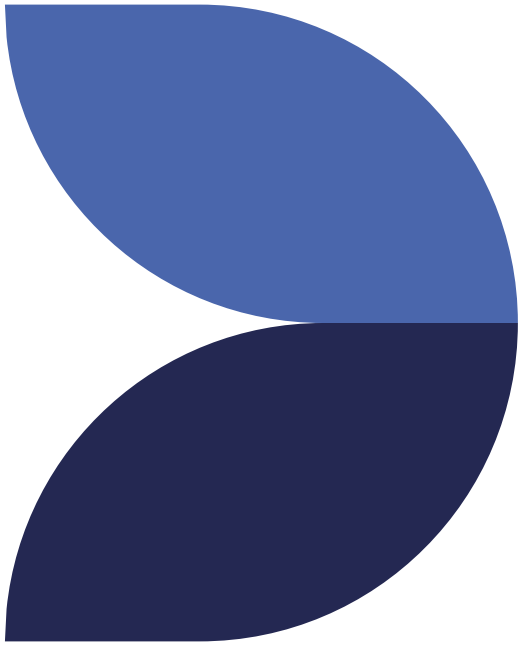
“Δ” captures alterations performed to the original system

Two major dimensions of alteration are: **change in functionality (*change in the business rules*)** and **change in implementation technique (*object-oriented language or a procedural language*).**

Contd..

- Legacy systems are the old sw systems which are essential for business process support.
- Software re-engineering is concerned with re-implementing legacy systems to make them more maintainable.

Legacy projects



- ***Legacy:*** any existing project that's difficult to maintain or extend.
  - project encompasses many other aspects, including
    - ✓ Build tools and scripts
    - ✓ Dependencies on other systems
    - ✓ The infrastructure on which the software runs
    - ✓ Project documentation
    - ✓ Methods of communication, such as between developers, or between developers and stakeholders
- Characteristics of legacy projects
  - Old:
    - number of generations of maintainers.
    - With each of these handoffs, knowledge about the original design of the system and the intentions of the previous maintainer is also lost
  - Large
    - Large projects are difficult and risky to replace, so they are more likely to live on and become legacy.

- Characteristics of legacy projects
  - Inherited:
    - people who originally wrote the code and those who now maintain it are not the same individuals
  - Poorly documented
    - Large projects are difficult and risky to replace, so they are more likely to live on and become legacy.

Just because a project fulfills some of the preceding criteria doesn't necessarily mean it should be treated as a legacy project.

- Linux kernel; in development since 1991, managed to maintain a very high level of quality.
- In 2012 after static analysis scan on the kernel it was found to have a defect density of 0.66 defects/kloc, which is lower than many commercial projects of a comparable size.

Legacy code

Untested, untestable code
Given that technical documentation for software projects is usually either nonexistent or unreliable, tests are often the best place to look for clues about the system's behavior and design assumptions. A good test suite can function as the de facto documentation for a project.

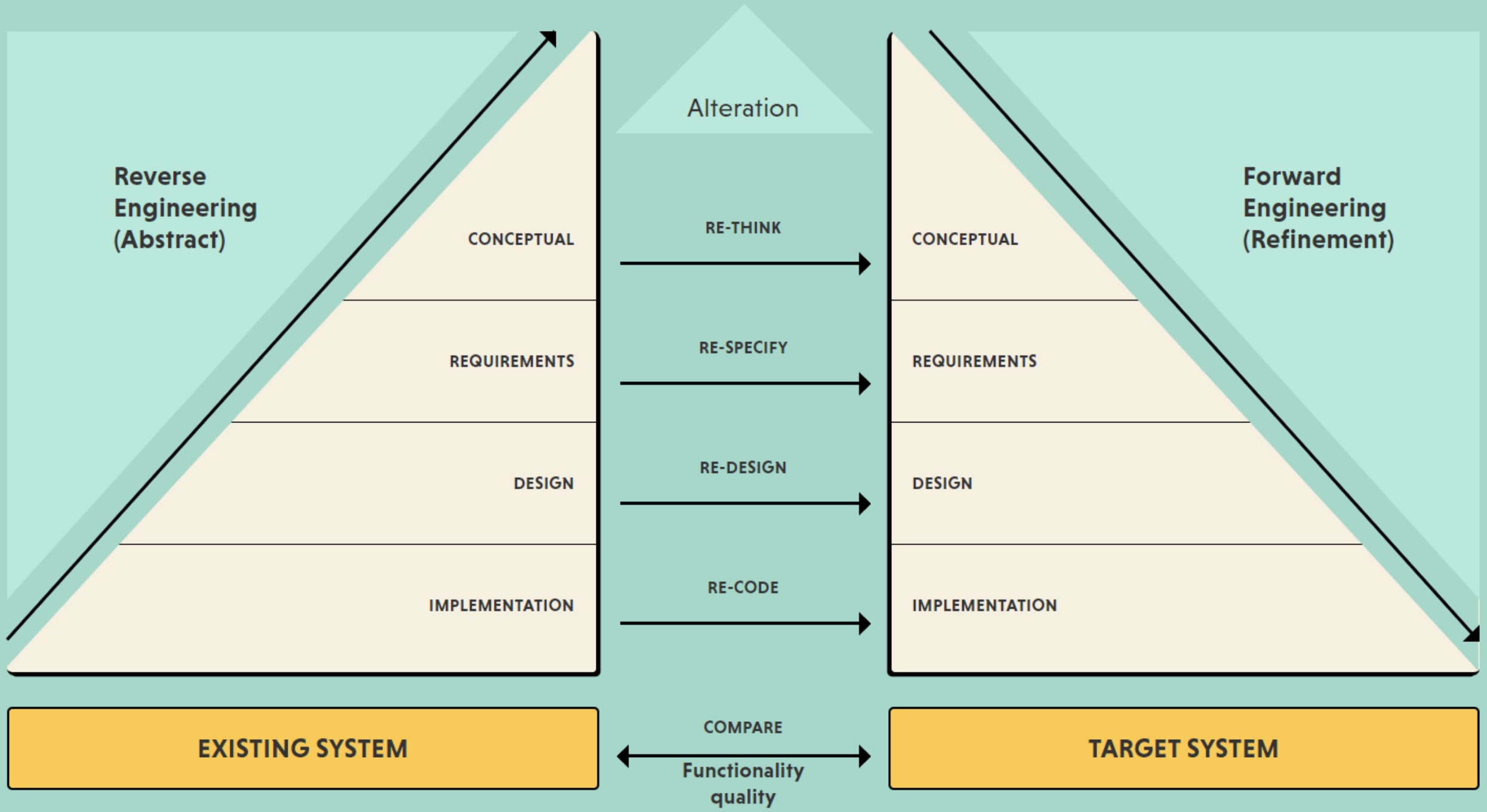
Inflexible code
A common problem with legacy code is that implementing new features or changes to existing behavior is inordinately difficult. What seems like a minor change can involve editing code in a lot of places. To make matters worse, each one of those edits also needs to be tested, often manually.

Code encumbered by technical debt
Every temporary or hacky solution reduces the overall quality of the project and makes future work more difficult. Implementing a quick-fix solution is analogous to taking out a loan, at some point this loan must be paid back. Until you repay the loan by refactoring and cleaning up the code, you'll be burdened with interest payments, meaning a codebase that's more difficult to work with.

Steps

Re-Engineering procedure requires the following steps

1. **Decide** which components of the software we want to re-engineer. Is it the complete software or just some components of the software?
2. **Do** Reverse Engineering to learn about existing software functionalities.
3. **Perform restructuring of source code** if needed for example modifying functional-Oriented programs in Object-Oriented programs
4. **Perform restructuring of data** if required
5. **Use Forward Engineering** ideas to generate re-engineered software


Alteration
Reverse Engineering (Abstract)
CONCEPTUAL
REQUIREMENTS
DESIGN
IMPLEMENTATION
RE-THINK
RE-SPECIFY
RE-DESIGN
RE-CODE
Forward Engineering (Refinement)
CONCEPTUAL
REQUIREMENTS
DESIGN
IMPLEMENTATION
EXISTING SYSTEM
COMPARE
Functionality quality
TARGET SYSTEM

Techniques used for re-engineering

In order to extract information that is not explicitly available in source code, automated analysis techniques, such as **lexical analysis, syntactic analysis, control flow analysis, data flow analysis,** and **program slicing** are used to facilitate reverse engineering.

Lexical Analysis is the first phase of the compiler also known as a scanner. It converts the High level input program into a sequence of **Tokens**.
The output is a sequence of tokens that is sent to the parser for syntax analysis
What is a token? A lexical token is a sequence of characters that can be treated as a unit in the grammar of the programming languages. **Example of tokens:**
•Type token (id, number, real, . . .)
•Punctuation tokens (IF, void, return, . . .)
•Alphabetic tokens (keywords)

Exercise 1: Count number of tokens :

```
int main()
{
  int a = 10, b = 20;
  printf("sum is :%d",a+b);
  return 0;
}
```

Answer: Total number of token: 27.

- Reverse engineering is done with the help of decompilers.
- A decompiler is a program that reads a program written in a machine language (the source language) and translates it into an equivalent program in a high level language (the target language)
- A decompiler, or reverse compiler, thus attempts to reverse the process of a compiler, which translates a highlevel language program into a binary or executable program

The **syntax analyzer** is the first phase of the decompiler. Its role is to group a sequence of bytes into a phrase or sentence of the language.
This sequence of bytes is checked for syntactic structure, that is, that the string belongs to the language.
Valid strings are represented by a parse tree, which is input into the next phase, the semantic analyzer.
The syntax analyzer is also known as the parser.
The **semantic analysis** phase determines the meaning of a group of machine instructions, collects information on the individual instructions of a subroutine,

Restructuring which is "the transformation from one representation form to another at the same relative abstraction level, while preserving the subject system's external behavior (function and semantics)"
For example, GOTOs are removed or software modules are modified to ensure one entry and one exit.

Reengineering a system involves not only the process or procedure side, but also the data side of an application system.
Process reengineering is a code-level procedure that analyzes control flow, transforming undesirable programming constructs into more efficient ones. Restructuring can play a major role in process reengineering.
Data reengineering examines and alters the data definitions, values and the use of data. Hard-coded parameters which are subject to change may be removed.

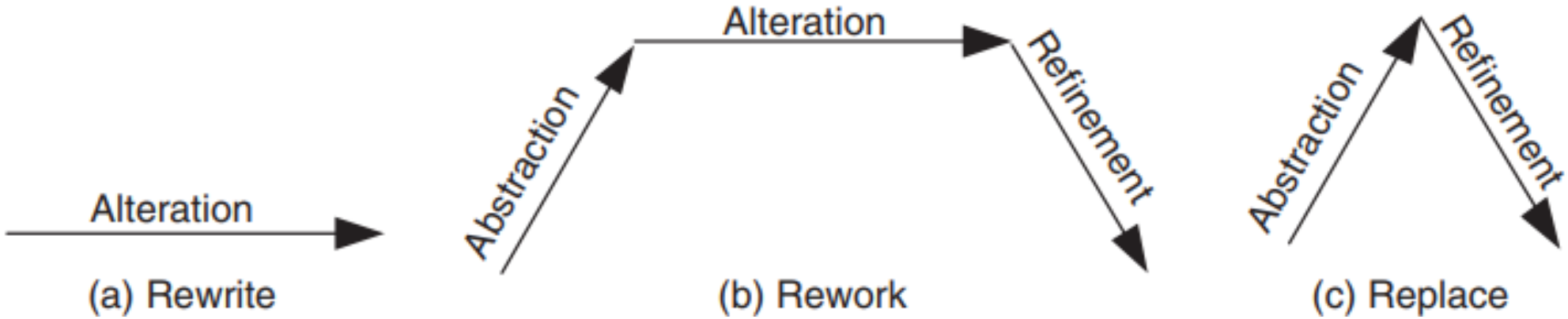
Software Reengineering Strategies



The **rewrite** strategy is based on the principle of alteration. **(Fortran code of a system can be rewritten in the C language)**

The **rework** strategy is based on the principles of abstraction, alteration, and refinement. **(replace the unstructured control flow constructs, namely GOTOs, with more commonly used structured constructs, say, a "for" loop)**

The **replace** strategy is based on the principles of abstraction and refinement. **(a program is represented at a higher level without using control flow concepts)**



Mandatory readings

- Chapter 1, Chris Birchall - Re-Engineering Legacy Software-Manning Publications (2016)
- Chapter 1, Priyadarshi Tripathy, Kshirasagar Naik - Software Evolution and Maintenance-Wiley (2014)

Thank you